

Module 07 — Neural Collaborative Filtering (PyTorch)

models/ncf_recommender.py • 609 lines

In one sentence

The deep learning model - and the single most important lesson in the project.

1. What this module does

This is the deep learning component the business case requires. It learns a vector (an 'embedding') for every customer and every product, then feeds those two vectors into a small neural network that predicts how much that customer would like that product.

The reason for using a neural network rather than SVD comes down to one thing. SVD scores a product as a **dot product** - a fixed, linear combination. But this data was built with a rule that is not linear: a customer buys when the category matches **and** the price fits. A dot product cannot express an 'and'. A neural network can learn one.

This module also contains the biggest mistake made during the project, and its fix. Read the negative sampling section carefully - it is the strongest thing you can talk about in a viva.

2. Concepts covered

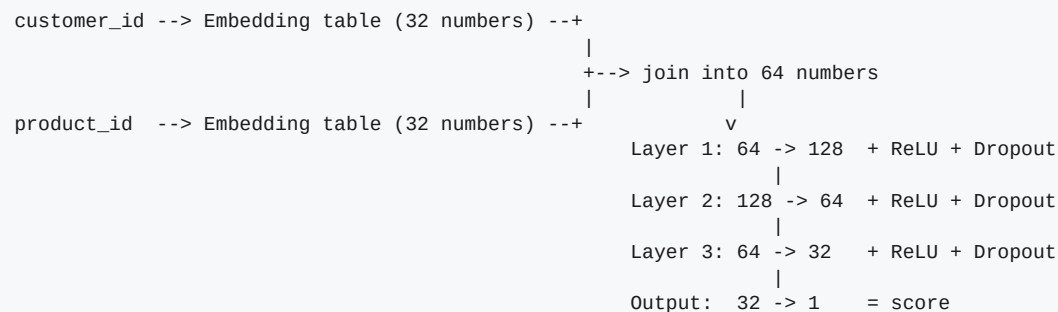
Every idea this module uses, in plain terms.

Concept	In simple terms	Where it is used here
Embeddings	A learned list of numbers representing a thing	32 numbers per customer and per product
Embedding lookup table	A big grid you index into by ID	nn.Embedding
MLP	A stack of layers that learn a function	128 to 64 to 32 to 1
ReLU activation	Keeps positives, zeroes negatives	Between MLP layers
Non-linearity	Lets the model learn 'and' rules, not just sums	The whole point of using an MLP
Dropout	Randomly ignore some neurons while training	Prevents memorising
Weight decay (L2)	Penalise large weights	Adam optimiser
Early stopping	Stop when validation stops improving	EarlyStopping class
MSE loss	Squared error, for predicting a number	Explicit / rating model
BCE loss	Log loss, for predicting yes/no	Implicit / engagement model
Negative sampling	Add examples of what was NOT chosen	The critical fix - see below
Mini-batch training	Train on 1,024 rows at a time	DataLoader
Xavier initialisation	Sensible starting weights	MLP layers
state_dict	Save the numbers, not the object	save_ncf_artifacts()
Train / valid / test split	Three separate data slices	Time-based, then random within train

3. How it works, step by step

- 1. Encode the IDs.** Customer 4,231 becomes row 892 in the embedding table. These mappings are saved, because serving must reproduce exactly the same index.
- 2. Split by time.** The last 90 days are held out completely. Training uses only what came before.
- 3. For the implicit model, sample negatives.** For every product a customer engaged with, pick 4 they never touched and label them 0.
- 4. Build the model.** Two embedding tables, concatenated, then a 3-layer MLP with ReLU and dropout, ending in a single score.
- 5. Train with Adam** in batches of 1,024, watching validation loss each epoch.
- 6. Stop early** when validation loss stops improving, and restore the best weights seen.
- 7. Save the weights** as a state_dict along with the architecture spec.
- 8. To recommend,** score every candidate product for that customer in a single batched forward pass and take the top N.

Architecture



4. Key functions

Function / class	What it does
encode_ids()	Maps real IDs to embedding row numbers
time_based_split()	Holds out the last 90 days
build_implicit_training_frame()	Adds sampled negatives - the critical function
InteractionsDataset	Wraps the data as PyTorch tensors
NeuralCollaborativeFiltering	The model itself
EarlyStopping	Watches validation loss, keeps the best weights
train_ncf_model()	The training loop
evaluate_explicit_model()	RMSE and MAE
evaluate_implicit_model()	Accuracy, precision, recall, F1
score_all_items()	Scores the whole catalogue in one pass
save_ncf_artifacts() / load_ncf_model()	Persistence

5. Inputs, outputs and how to run

	Detail
Inputs	interactions.csv
Outputs	ncf_model_explicit.pt, ncf_model_implicit.pt, index maps, histories
Parameters	278,817
Results	Explicit RMSE 0.9108 • Implicit accuracy 0.8583, F1 0.5816
Runtime	About 8 minutes on CPU
Run with	python models/ncf_recommender.py

6. Key code

models/ncf_recommender.py — build_implicit_training_frame()

```

118 | def build_implicit_training_frame(encoded_df: pd.DataFrame, num_items: int,
119 |                               num_negatives: int = NUM_NEGATIVES,
120 |                               seed: int = RANDOM_SEED) -> pd.DataFrame:
121 |     """Build the implicit training set with sampled negatives."""
122 |     # Without negatives the model is asked "given they clicked, did they buy?"
123 |     # - a conversion problem. Retrieval needs contrast against the catalogue.
124 |     # Measured: 53% accuracy / NDCG 0.0000 without, 86% / 0.0576 with.
125 |     rng = np.random.default_rng(seed)
126 |
127 |     positives = encoded_df.loc[encoded_df["implicit_feedback"] == 1, ["user_idx", "item_idx"]]
128 |
129 |     # Exclude everything touched, including viewed-not-carted: those are
130 |     # ambiguous, not confirmed negatives.
131 |     seen_by_user = encoded_df.groupby("user_idx")["item_idx"].apply(set)
132 |
133 |     negative_users = []
134 |     negative_items = []
135 |
136 |     for user_idx, group in positives.groupby("user_idx"):
137 |         required = len(group) * num_negatives
138 |         seen = seen_by_user.get(user_idx, set())
139 |
140 |         candidates = rng.integers(0, num_items, size=int(required * 1.5) + 16)
141 |         sampled = [int(c) for c in candidates if c not in seen][:required]
142 |
143 |         negative_users.extend([user_idx] * len(sampled))
144 |         negative_items.extend(sampled)
145 |
146 |     positive_frame = pd.DataFrame({
147 |         "user_idx": positives["user_idx"].to_numpy(),
148 |         ...

```

The most important function in the deep learning component.

models/ncf_recommender.py — NeuralCollaborativeFiltering.forward()

```

232 |     def forward(self, user_indices, item_indices):
233 |         """Score a batch of (user, item) pairs."""
234 |         # Concatenation, not element-wise product: a product hard-codes a
235 |         # multiplicative interaction and reduces this to matrix factorisation.
236 |         user_vec = self.user_embedding(user_indices)
237 |         item_vec = self.item_embedding(item_indices)
238 |
239 |         x = torch.cat([user_vec, item_vec], dim=1)
240 |         x = self.mlp(x)
241 |         return self.output_layer(x).squeeze(1)

```

Note torch.cat - joining the two vectors, not multiplying them.

7. Things to remember

The biggest mistake in the project: no negative examples

The first version trained the implicit model on the interaction log alone, labelling each row by whether it converted. But **every row in the log is a product the platform already chose to show**. So the model was being asked 'given they clicked it, did they buy it?' - a conversion question. The real question a recommender must answer is 'out of 2,500 products, which would this customer engage with at all?' - and the log contains **no examples of products they would not engage with**. Result: 53% accuracy, barely better than a coin flip, and NDCG@10 of 0.0000.

The fix and what it bought

For every product a customer engaged with, sample 4 products they never touched and label those 0. Now telling 'would engage' from 'would not' is the training objective. Accuracy went from 0.534 to **0.858**, F1 from 0.350 to **0.582**, and NDCG@10 from 0.0000 to **0.0576**. Same architecture, same data - only the training examples changed.

A good RMSE does not mean a good recommender

The explicit model predicts star ratings well: RMSE 0.91 on a 1-5 scale. It is also **completely useless for ranking**, scoring $\text{NDCG@10} = 0.0000$ and returning the same 14 products to every single customer. It had never been shown a product a customer did *not* interact with, so scoring unseen products is outside anything it learned. This is why production ranks with the implicit model.

Why concatenate the embeddings instead of multiplying them

Multiplying them element by element hard-codes a multiplicative relationship - which is exactly what matrix factorisation already does, so the neural network would add nothing. Joining them end to end leaves the relationship itself for the network to learn.

8. Likely questions

Q. What is an embedding, in plain terms?

A. A list of 32 numbers the model learns for each customer and each product. Nobody chooses what the numbers mean - training works them out. Customers with similar taste end up with similar numbers.

Q. Why use a neural network when SVD already factorises the matrix?

A. SVD combines its latent factors by a dot product, which is a fixed linear sum. This data has a conjunctive rule - buy when the category matches AND the price fits. A sum cannot express an AND; a neural network can learn one.

Q. Your NCF scored 0.0000 at first. What happened?

A. I was using the explicit rating model for ranking. It only ever saw products customers had interacted with, so scoring the rest of the catalogue was outside its training distribution and it fell back on a global item bias - the same for everybody. Switching to the implicit model trained with negative sampling fixed it.

Q. Why does early stopping restore the best weights?

A. Otherwise you stop at the right time but keep the weights from the last epoch, which is by definition worse than the best one. That would defeat the purpose of watching validation loss at all.

Q. Why save a state_dict rather than the model?

A. Saving the whole object pickles the class definition with it, so any later edit to the file breaks loading. A state_dict is just the numbers, and it loads into a freshly built model of the same shape.